

PROGRAMME OUTCOMES (PO)

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development (WK1 to WK4).
PO3	Design / Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required (WK5)
PO4	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.
PO5	Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations
PO6	The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development
PO8	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws (WK9)
PO9	Individual and Team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings
PO 10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions
PO11	Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change

PROGRAMME SPECIFIC OUTCOMES (PSO)

Upon completion of the programme, graduates will have ability to:

- **PSO1** Graduates will demonstrate multidisciplinary knowledge for problem solving by creating solutions for product based and application-based software for the advancement of the society.
- **PSO2** Graduates attain advance knowledge in Information and Communication Technologies (ICT) thereby creating real time solutions for different projects by using modern tools prevailing in the current trends.
- **PSO3** Graduates will exhibit state of the art technologies by applying their knowledge in various programming skills to overcome the demand of sus

23CSC04 – PRINCIPLES OF COMPILER DESIGN LABORATORY

(LAB COMPONENT) LIST OF EXPERIMENTS

Exp. No.	Date	Name of the Experiment	Page No.
1.	07-07-26	Implementation of lexical analyzer using C and LEX	1 – 6
2.	13-07-26	Implementation of a calculator that takes an expression (with digits, + and *), computes and print its value, using YACC.	7 – 9
3.	25-07-26	Implementation of a parser using LEX and YACC.	10 – 12
4.	31-07-26	Implementation of symbol table.	13 – 16
5.	06-08-26	Implementation of Predictive parsing.	17 – 23
6.	12-08-26	Implementation of Shift Reduce Parsing Algorithm.	24 – 28
7.	20-08-26	Implementation of LR parsing.	29 – 36
8.	27-08-26	Implementation of front end of a compiler that generates the three-address code for a simple language with One data type integer, arithmetic operators, relational operators, variable declaration statement, one conditional construct, one iterative construct and assignment statement.	37 – 41
9.	02-09-26	Implementation of back end of the compiler which takes the three-address code as input and produces assembly language instructions that can be assembled and run using an 8086 assembler. The target assembly instructions can be simple move, add, sub, and jump.	42 – 45
10.	10-09-26	Implementation of the code optimizer phase of a compiler that eliminates dead code and common sub-expressions.	46 - 48

Signature of the Faculty

RUBRICS REFERENCE TABLE

Criteria	Range of Marks			
	Excellent	Good	Average	Below Average
Aim & Algorithm (20 marks)	19-20	17-18	15-16	0-14
Program with Proper Syntax and Structure (30 Marks)	27-30	21-26	15-20	0-14
Compilation and Debugging (30 Marks)	27-30	21-26	15-20	0-14
Documentation (10 Marks)	9-10	7-8	5-6	0-4
Viva (10 Marks)	9-10	7-8	5-6	0-4
Overall Marks	90-100	70-89	50-69	0-49

Components	EX1	EX2	EX3	EX4	EX5	EX6	EX7	EX8	EX9	EX10
Aim & Algorithm (20 marks)										
Program with Proper Syntax and Structure (30 Marks)										
Compilation and Debugging (30 Marks)										
Documentation (10 Marks)										
Viva (10 Marks)										
TOTAL										
Consolidated Mark (100)										
Faculty Signature										

Exp No: 1(a)	Implementation of lexical analyzer using C programming
Date: 07-07-26	

Aim:

To implement a Lexical Analyzer using C programming that reads an input program and identifies different types of tokens such as keywords, identifiers, constants, operators, and special symbols.

Algorithm:

1. Start the program.
2. Read the input C program character by character.
3. Ignore spaces, tabs, and newlines.
4. If the character is an alphabet or _, collect characters until a non-alphanumeric character is found.
 - i. Check whether the collected word is a keyword.
 - ii. If not, identify it as an identifier.
5. If the character is a digit, collect all consecutive digits and identify them as a constant.
6. If the character is an operator such as +, -, *, /, =, <, >, identify it as an operator.
7. If the character is a special symbol such as (,), {, }, :, ,, identify it as a special symbol.
8. Continue until the end of the input.
9. Display all identified tokens.
10. Stop.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char str[])
{
    char keywords[32][10] = {
        "auto", "break", "case", "char", "const", "continue",
        "default", "do", "double", "else", "enum", "extern",
        "float", "for", "goto", "if", "int", "long",
        "register", "return", "short", "signed", "sizeof", "static",
        "struct", "switch", "typedef", "union", "unsigned", "void",
        "volatile", "while"
    };

    int i;

    for (i = 0; i < 32; i++)
    {
        if (strcmp(str, keywords[i]) == 0)
            return 1;
    }

    return 0;
}

int main()
{
```

```

char input[1000];
int i = 0;

printf("Enter the C program:\n");
fgets(input, sizeof(input), stdin);

printf("\nTokens:\n");

while (input[i] != '\0')
{
    /* Ignore spaces and newlines */
    if (isspace(input[i]))
    {
        i++;
    }

    /* Identifier or Keyword */
    else if (isalpha(input[i]) || input[i] == '_')
    {
        char word[50];
        int j = 0;

        while (isalnum(input[i]) || input[i] == '_')
        {
            word[j++] = input[i++];
        }

        word[j] = '\0';

        if (isKeyword(word))
            printf("%s -> Keyword\n", word);
        else
            printf("%s -> Identifier\n", word);
    }

    /* Constant */
    else if (isdigit(input[i]))
    {
        char number[50];
        int j = 0;

        while (isdigit(input[i]))
        {
            number[j++] = input[i++];
        }

        number[j] = '\0';

        printf("%s -> Constant\n", number);
    }

    /* Operators */
    else if (input[i] == '+' || input[i] == '-' ||
             input[i] == '*' || input[i] == '/' ||
             input[i] == '=' || input[i] == '<' ||

```

```

        input[i] == '>')
    {
        printf("%c -> Operator\n", input[i]);
        i++;
    }

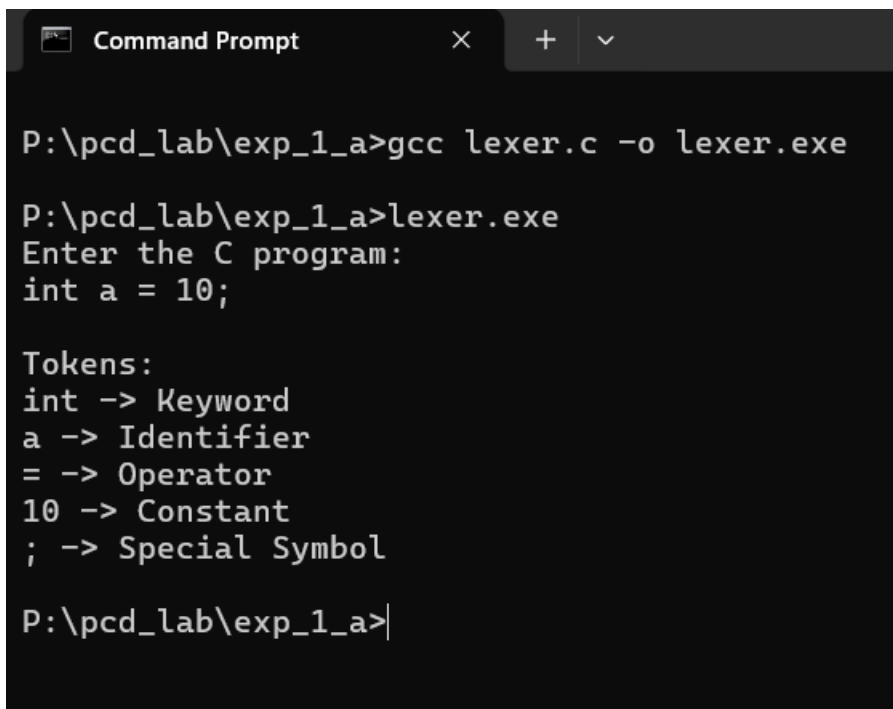
    /* Special Symbols */
    else if (input[i] == '(' || input[i] == ')' ||
             input[i] == '{' || input[i] == '}' ||
             input[i] == '[' || input[i] == ']' ||
             input[i] == ';' || input[i] == ',')
    {
        printf("%c -> Special Symbol\n", input[i]);
        i++;
    }

    /* Unknown character */
    else
    {
        printf("%c -> Unknown Symbol\n", input[i]);
        i++;
    }
}

return 0;
}

```

Sample Output:



```
Command Prompt
P:\pcd_lab\exp_1_a>gcc lexer.c -o lexer.exe
P:\pcd_lab\exp_1_a>lexer.exe
Enter the C program:
int a = 10;

Tokens:
int -> Keyword
a -> Identifier
= -> Operator
10 -> Constant
; -> Special Symbol
P:\pcd_lab\exp_1_a>
```

Result:

Thus, the Lexical Analyzer was successfully implemented in C, and the input program was successfully divided into different tokens such as keywords, identifiers, constants, operators, and special symbols.

Exp No: 1(b)	Implementation of Lexical analyzer using LEX
Date: 07-07-26	

Aim:

To implement a Lexical Analyzer using LEX to identify keywords, identifiers, numbers, operators, and special symbols from the given input.

Algorithm:

1. Start the program.
2. Create a LEX specification file with the extension .l.
3. Define regular expressions for keywords, identifiers, numbers, operators, and special symbols.
4. Define actions to display the type of each recognized token.
5. Save the LEX program as lexer.l.
6. Open Command Prompt and navigate to the folder containing lexer.l.
7. Execute flex lexer.l to generate the C source file lex.yy.c.
8. Compile the generated C file using gcc lex.yy.c -o lexer.exe.
9. Execute the generated program using lexer.exe.
10. Enter the input statement.
11. The lexical analyzer identifies and displays each token.
12. Stop the program.

Program:

```
%{
#include <stdio.h>
%}

%%

"int"|"float"|"char"|"if"|"else"|"for"|"while"|"return"
{ printf("%s -> Keyword\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]*
{ printf("%s -> Identifier\n", yytext); }

[0-9]+
{ printf("%s -> Number\n", yytext); }

"+"|"-"|"*"|"/"|"="|"<"|">"
{ printf("%s -> Operator\n", yytext); }

"("|")"|"{"|"}"|";"|"|"
{ printf("%s -> Special Symbol\n", yytext); }

[ \t\n]+
{ }

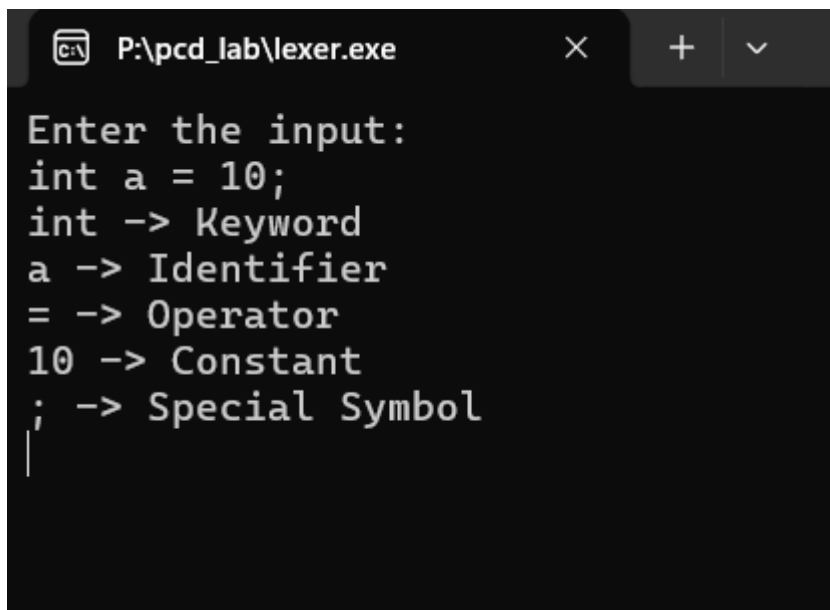
.
{ printf("%s -> Unknown Symbol\n", yytext); }

%%
```

```
int yywrap()
{
    return 1;
}

int main()
{
    printf("Enter the input:\n");
    yylex();
    return 0;
}
```

Sample Output:

A screenshot of a Windows command prompt window titled "P:\pcd_lab\lexer.exe". The window has standard Windows window controls (close, maximize, and a dropdown arrow). The text inside the window shows the program's output: "Enter the input:" followed by the input "int a = 10;". The program then outputs the following tokenization results: "int -> Keyword", "a -> Identifier", "= -> Operator", "10 -> Constant", and "; -> Special Symbol". A vertical cursor is visible on the line following the semicolon token.

```
Enter the input:
int a = 10;
int -> Keyword
a -> Identifier
= -> Operator
10 -> Constant
; -> Special Symbol
|
```

Result:

Thus, the Lexical Analyzer was successfully implemented using LEX, and the given input was successfully analyzed into keywords, identifiers, numbers, operators, and special symbols.

Exp No: 2	Implementation of a Calculator that takes an expression (with digit + & *) compute and print its value using YACC tool
Date: 13-07-26	

Aim:

To implement a simple calculator using YACC that accepts an arithmetic expression containing digits, addition (+) and multiplication (*) operators, evaluates the expression according to operator precedence, and displays the result.

Algorithm:

1. Start the program.
2. Create a Lex file to recognize digits, +, *, and newline.
3. Create a YACC file containing grammar rules for the arithmetic expression.
4. Define the precedence of operators, with * having higher precedence than +.
5. Save the Lex file as calc.l and the YACC file as calc.y.
6. Generate the Lex C source file using flex calc.l.
7. Generate the YACC parser using bison -dy calc.y.
8. Compile the generated C files using GCC.
9. Execute the generated calculator program.
10. Enter an arithmetic expression containing digits, +, and *.
11. YACC parses the expression and calculates its value.
12. Display the calculated result.
13. Stop the program.

Lex Program — calc.l :

```
%{
#include "y.tab.h"
#include <stdlib.h>
}%

%%

[0-9]+ { yylval = atoi(yytext); return NUMBER; }
"+"    { return '+'; }
"*"    { return '*'; }
"\n"   { return '\n'; }
[ \t]  { }

.      { return yytext[0]; }

%%

int yywrap()
{
    return 1;
}
```

YACC Program — calc.y :

```
%{
#include <stdio.h>
#include <stdlib.h>
```

```

int yylex();
int yyerror(char *s);
%}

%token NUMBER

%left '+'
%left '*'

%%

input:
    expr '\n'    { printf("Result = %d\n", $1); }
    ;

expr:
    expr '+' expr { $$ = $1 + $3; }
  | expr '*' expr { $$ = $1 * $3; }
  | NUMBER       { $$ = $1; }
    ;

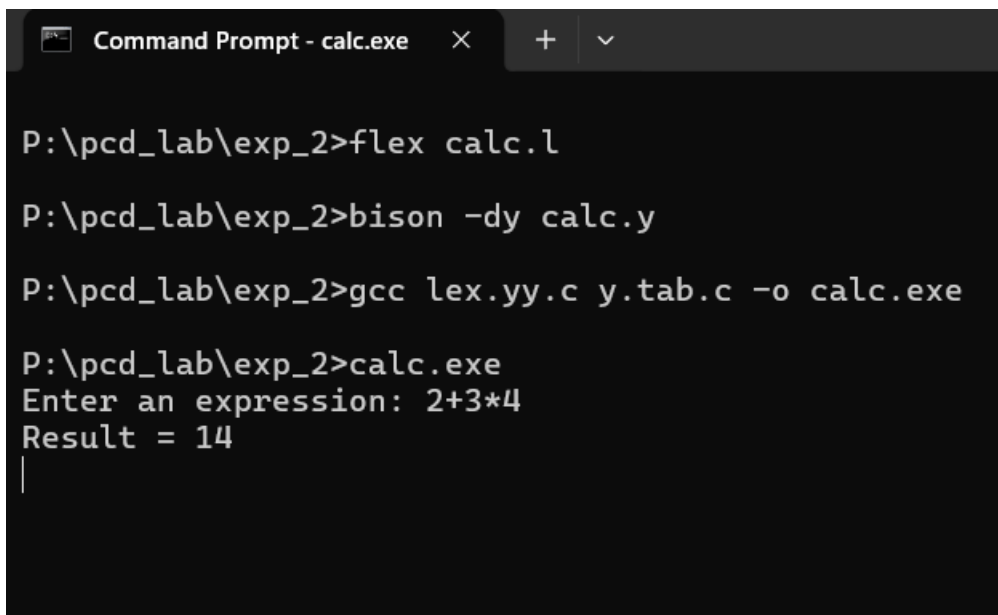
%%

int yyerror(char *s)
{
    printf("Invalid expression\n");
    return 0;
}

int main()
{
    printf("Enter an expression: ");
    yyparse();
    return 0;
}

```

Sample Output:



```
P:\pcd_lab\exp_2>flex calc.l
P:\pcd_lab\exp_2>bison -dy calc.y
P:\pcd_lab\exp_2>gcc lex.yy.c y.tab.c -o calc.exe
P:\pcd_lab\exp_2>calc.exe
Enter an expression: 2+3*4
Result = 14
|
```

Result:

Thus, a simple calculator was successfully implemented using LEX and YACC, which accepts arithmetic expressions containing digits, +, and *, evaluates them according to operator precedence, and displays the result.

Exp No: 3	Implementation of Parser using LEX and YACC tool
Date: 25-07-26	

Aim:

To implement a parser using LEX and YACC that checks whether the given input string follows the specified grammar and determines whether the input is valid or invalid.

Algorithm:

1. Start the program.
2. Create a Lex file to recognize identifiers and operators.
3. Create a YACC file containing grammar rules for valid expressions.
4. Save the Lex file as parser.l.
5. Save the YACC file as parser.y.
6. Open Command Prompt and navigate to the folder containing the files.
7. Execute flex parser.l to generate lex.yy.c.
8. Execute bison -dy parser.y to generate y.tab.c and y.tab.h.
9. Compile the generated C files using GCC.
10. Execute the generated parser.exe file.
11. Enter an expression.
12. YACC checks the expression according to the grammar rules.
13. If the expression follows the grammar, display Valid Expression.
14. Otherwise, display Invalid Expression.
15. Stop the program.

Lex Program — parser.l

```
%{
#include "y.tab.h"
}%

%%

[a-zA-Z][a-zA-Z0-9_]* { return ID; }
"+" { return '+'; }
"*" { return '*'; }
"(" { return '('; }
")" { return ')'; }
"\n" { return '\n'; }
[ \t] { }

. { return yytext[0]; }

%%

int yywrap()
{
    return 1;
}
```

YACC Program — parser.y

```
%{
#include <stdio.h>
```

```

int yylex();
int yyerror(char *s);
%}

%token ID

%%

input:
    expr '\n'  { printf("Valid Expression\n"); }
    ;

expr:
    expr '+' term
    | term
    ;

term:
    term '*' factor
    | factor
    ;

factor:
    ID
    | '(' expr ')'
    ;

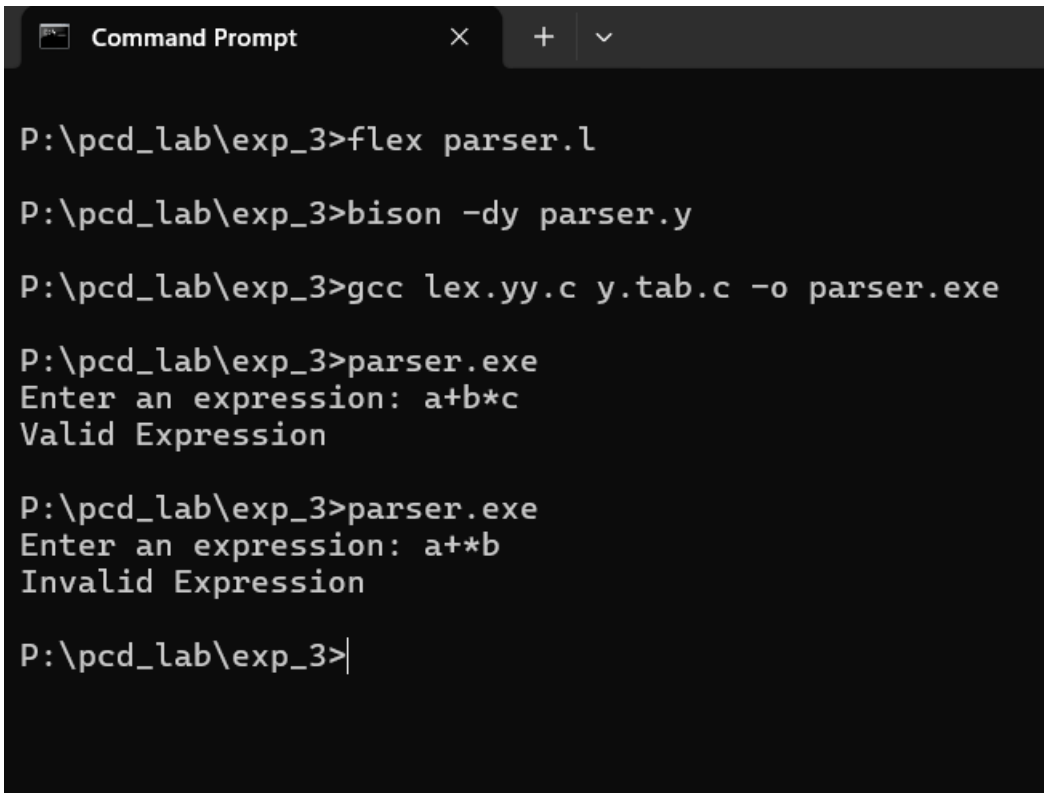
%%

int yyerror(char *s)
{
    printf("Invalid Expression\n");
    return 0;
}

int main()
{
    printf("Enter an expression: ");
    yyparse();
    return 0;
}

```

Sample Output:



```
P:\pcd_lab\exp_3>flex parser.l
P:\pcd_lab\exp_3>bison -dy parser.y
P:\pcd_lab\exp_3>gcc lex.yy.c y.tab.c -o parser.exe
P:\pcd_lab\exp_3>parser.exe
Enter an expression: a+b*c
Valid Expression

P:\pcd_lab\exp_3>parser.exe
Enter an expression: a+b
Invalid Expression

P:\pcd_lab\exp_3>|
```

Result:

Thus, a parser was successfully implemented using LEX and YACC, and the parser successfully identified whether the given input expression was valid or invalid according to the specified grammar.

Exp No: 4	Implementation of Symbol Table
Date: 31-07-26	

Aim:

To implement a symbol table using C programming that stores the symbol, address, data type, size, and offset of variables declared in declaration statements.

Algorithm:

1. Start the program.
2. Define a structure to store symbol, address, data type, size, and offset.
3. Read the number of declaration statements.
4. Read each declaration.
5. Extract the data type and variable names.
6. Determine the size of each data type.
7. Assign an offset to each variable starting from 0.
8. Calculate the address using the base address and offset.
9. Store all information in the symbol table.
10. Display the complete symbol table.
11. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
struct Symbol
{
    char name[20];
    unsigned long address;
    char type[20];
    int size;
    int offset;
};
```

```
struct Symbol table[50];
int count = 0;
int offset = 0;
```

```
int getSize(char type[])
{
    if (strcmp(type, "int") == 0)
        return 4;

    if (strcmp(type, "float") == 0)
        return 4;

    if (strcmp(type, "char") == 0)
        return 1;

    if (strcmp(type, "double") == 0)
        return 8;
```

```

    return 0;
}

void processDeclaration(char line[])
{
    char type[20];
    char *token;
    char *rest;

    token = strtok(line, " \t");

    if (token == NULL)
        return;

    strcpy(type, token);

    rest = strtok(NULL, "");

    if (rest == NULL)
        return;

    token = strtok(rest, ",");

    while (token != NULL)
    {
        char name[20];
        int i = 0, j = 0;
        int size;

        while (token[i] != '\0')
        {
            if (!isspace((unsigned char)token[i]))
                name[j++] = token[i];

            i++;
        }

        name[j] = '\0';

        if (j > 0 && name[j - 1] == ';')
            name[j - 1] = '\0';

        if (strlen(name) > 0)
        {
            size = getSize(type);

            strcpy(table[count].name, name);
            strcpy(table[count].type, type);

            table[count].size = size;
            table[count].offset = offset;
            table[count].address = 1000 + offset;

            offset = offset + size;
        }
    }
}

```

```

        count++;
    }

    token = strtok(NULL, ",");
}
}

int main()
{
    int n, i;
    char line[100];

    printf("Enter the number of declarations: ");
    scanf("%d", &n);

    getchar();

    printf("\nEnter the declarations:\n");

    for (i = 0; i < n; i++)
    {
        fgets(line, sizeof(line), stdin);
        processDeclaration(line);
    }

    printf("\n----- SYMBOL TABLE ----- \n");

    printf("%-12s %-12s %-12s %-8s %-8s\n",
           "Symbol", "Address", "Type", "Size", "Offset");

    printf("----- \n");

    for (i = 0; i < count; i++)
    {
        printf("%-12s %-12lu %-12s %-8d %-8d\n",
               table[i].name,
               table[i].address,
               table[i].type,
               table[i].size,
               table[i].offset);
    }

    return 0;
}

```

Sample Output:

```
Command Prompt
P:\pcd_lab\exp_4>gcc symbol.c -o symbol.exe

P:\pcd_lab\exp_4>symbol.exe
Enter the number of declarations: 2

Enter the declarations:
int a,b
float c,d

----- SYMBOL TABLE -----
Symbol      Address      Type      Size      Offset
-----
a           1000         int        4          0
b           1004         int        4          4
c           1008         float       4          8
d           1012         float       4         12

P:\pcd_lab\exp_4>
```

Result:

Thus, the Symbol Table was successfully implemented in C, and the variables declared were stored with their symbol, address, data type, size, and offset.

Exp No: 5	Implementation of Predictive Parser
Date: 06-08-26	

Aim:

To implement a Predictive Parser using C programming for a given context-free grammar and determine whether the input string is accepted or rejected using a parsing table.

Grammar Used

The following grammar is used for predictive parsing:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow (E) \mid id$

Here, id represents an identifier.

Algorithm:

1. Start the program.
2. Define the grammar and predictive parsing table.
3. Initialize the stack with \$ and the start symbol E.
4. Append \$ to the end of the input string.
5. Compare the top of the stack with the current input symbol.
6. If both are the same terminal, pop the stack and move to the next input symbol.
7. If the top of the stack is a non-terminal, use the predictive parsing table to select the appropriate production.
8. Replace the non-terminal on the stack with the symbols on the right-hand side of the selected production.
9. If no production is available, reject the input string.
10. Continue until both the stack and input contain \$.
11. If both match, accept the input string.
12. Stop the program.

Program:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char stack[100];
```

```
int top = -1;
```

```
void push(char c)
```

```
{
    stack[++top] = c;
}
```

```
char pop()
```

```
{
    return stack[top--];
}
```

```
void printStack()
```

```
{
    int i;
```

```

    for (i = 0; i <= top; i++)
        printf("%c", stack[i]);
}

int main()
{
    char input[100];
    int ip = 0;
    char X;

    printf("Grammar:\n");
    printf("E -> TR\n");
    printf("R -> +TR | e\n");
    printf("T -> FS\n");
    printf("S -> *FS | e\n");
    printf("F -> (E) | i\n");

    printf("Enter the input string: ");
    scanf("%s", input);

    strcat(input, "$");

    push('$');
    push('E');

    printf("\n%-15s %-15s Action\n", "Stack", "Input");
    printf("-----\n");

    while (top >= 0)
    {
        X = stack[top];

        /* Print current stack and input */
        printf("%-15s %-15s ", "", "");
        printf("\r");

        printStack();
        printf("%*s", 15 - top - 1, "");

        printf("%-15s ", &input[ip]);

        if (X == '$' && input[ip] == '$')
        {
            printf("Accept\n");
            printf("\nInput string is ACCEPTED.\n");
            break;
        }

        /* Matching terminal */
        if (X == input[ip])
        {
            pop();
            ip++;
        }
    }
}

```

```

    printf("Match %c\n", X);
}

/* E -> TR */
else if (X == 'E')
{
    pop();

    if (input[ip] == 'i' || input[ip] == '(')
    {
        push('R');
        push('T');

        printf("E -> TR\n");
    }
    else
    {
        printf("Error\n");
        printf("\nInput string is REJECTED.\n");
        break;
    }
}

/* R -> +TR or e */
else if (X == 'R')
{
    pop();

    if (input[ip] == '+')
    {
        push('R');
        push('T');
        push('+');

        printf("R -> +TR\n");
    }
    else if (input[ip] == ')' || input[ip] == '$')
    {
        printf("R -> e\n");
    }
    else
    {
        printf("Error\n");
        printf("\nInput string is REJECTED.\n");
        break;
    }
}

/* T -> FS */
else if (X == 'T')
{
    pop();

    if (input[ip] == 'i' || input[ip] == '(')
    {

```

```

        push('S');
        push('F');

        printf("T -> FS\n");
    }
    else
    {
        printf("Error\n");
        printf("\nInput string is REJECTED.\n");
        break;
    }
}

/* S -> *FS or e */
else if (X == 'S')
{
    pop();

    if (input[ip] == '*')
    {
        push('S');
        push('F');
        push('*');

        printf("S -> *FS\n");
    }
    else if (input[ip] == '+' ||
             input[ip] == ')' ||
             input[ip] == '$')
    {
        printf("S -> e\n");
    }
    else
    {
        printf("Error\n");
        printf("\nInput string is REJECTED.\n");
        break;
    }
}

/* F -> i or (E) */
else if (X == 'F')
{
    pop();

    if (input[ip] == 'i')
    {
        push('i');

        printf("F -> i\n");
    }
    else if (input[ip] == '(')
    {
        push('(');
        push('E');
    }
}

```

```
        push('(');

        printf("F -> (E)\n");
    }
    else
    {
        printf("Error\n");
        printf("\nInput string is REJECTED.\n");
        break;
    }
}

else
{
    printf("Error\n");
    printf("\nInput string is REJECTED.\n");
    break;
}
}

return 0;
}
```

Sample Output:

```
Command Prompt
R -> +TR | e
T -> FS
S -> *FS | e
F -> (E) | i

Enter the input string: i+i*i

Stack      Input      Action
-----
$E          i+i*i$    E -> TR
$RT         i+i*i$    T -> FS
$RSF        i+i*i$    F -> i
$RSi        i+i*i$    Match i
$RS         +i*i$     S -> e
$R          +i*i$     R -> +TR
$RT+        +i*i$     Match +
$RT         i*i$      T -> FS
$RSF        i*i$      F -> i
$RSi        i*i$      Match i
$RS         *i$      S -> *FS
$RSF*       *i$      Match *
$RSF        i$       F -> i
$RSi        i$       Match i
$RS         $        S -> e
$R          $        R -> e
$           $        Accept

Input string is ACCEPTED.

P:\pcd_lab\exp_5>
```

```
Command Prompt
P:\pcd_lab\exp_5>gcc predictive.c -o predictive.exe

P:\pcd_lab\exp_5>predictive.exe
Grammar:
E -> TR
R -> +TR | e
T -> FS
S -> *FS | e
F -> (E) | i

Enter the input string: i*i+

Stack      Input      Action
-----
$E          i*i+$      E -> TR
$RT          i*i+$      T -> FS
$RSF        i*i+$      F -> i
$RSi        i*i+$      Match i
$RS         *i+$      S -> *FS
$RSF*       *i+$      Match *
$RSF        i+$      F -> i
$RSi        i+$      Match i
$RS         +$      S -> e
$R          +$      R -> +TR
$RT+        +$      Match +
$RT         $       Error

Input string is REJECTED.

P:\pcd_lab\exp_5>
```

Result:

Thus, the Predictive Parser was successfully implemented using C programming, and the given input string was successfully checked according to the specified grammar. Valid strings were accepted and invalid strings were rejected.

Exp No: 6	Implementation of Shift Reduce Parser
Date: 12-08-26	

Aim:

To implement the Shift Reduce Parsing Algorithm using C programming to determine whether a given input string is accepted or rejected according to a specified grammar.

Grammar Used

The following grammar is used:

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

Here, i is used to represent id .

For example:

$i+i*i$

represents:

$id + id * id$

Algorithm:

1. Start the program.
2. Initialize an empty stack and append \$ to the input string.
3. Read the input string from left to right.
4. Shift the next input symbol onto the stack.
5. Check whether the top of the stack contains a handle that can be reduced.
6. If i is found, reduce it to E .
7. If $E+E$ is found, reduce it to E .
8. If $E * E$ is found, reduce it to E .
9. Continue shifting and reducing until the entire input is processed.
10. If the stack contains $\$ \$$, accept the input.
11. Otherwise, reject the input.
12. Stop the program.

Program:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char stack[100];
```

```
char input[100];
```

```
int top = -1;
```

```
int ip = 0;
```

```
void shift()
```

```
{
    stack[++top] = input[ip++];
    stack[top + 1] = '\0';
```

```
    printf("%-15s %-15s SHIFT\n", stack, &input[ip]);
}
```

```
int reduce()
```

```
{
    /* Reduce i to E */
```

```

if (top >= 0 && stack[top] == 'i')
{
    stack[top] = 'E';
    stack[top + 1] = '\0';

    printf("%-15s %-15s REDUCE E->id\n",
        stack, &input[ip]);

    return 1;
}

/* Reduce E+E to E */
if (top >= 2 &&
    stack[top - 2] == 'E' &&
    stack[top - 1] == '+' &&
    stack[top] == 'E')
{
    top -= 2;
    stack[top] = 'E';
    stack[top + 1] = '\0';

    printf("%-15s %-15s REDUCE E->E+E\n",
        stack, &input[ip]);

    return 1;
}

/* Reduce E*E to E */
if (top >= 2 &&
    stack[top - 2] == 'E' &&
    stack[top - 1] == '*' &&
    stack[top] == 'E')
{
    top -= 2;
    stack[top] = 'E';
    stack[top + 1] = '\0';

    printf("%-15s %-15s REDUCE E->E*E\n",
        stack, &input[ip]);

    return 1;
}

return 0;
}

int main()
{
    printf("Grammar:\n");
    printf("E -> E+E\n");
    printf("E -> E*E\n");
    printf("E -> id\n");

    printf("Enter the input string: ");
    scanf("%s", input);

```

```

    strcat(input, "$");

    stack[++top] = '$';
    stack[top + 1] = '\0';

    printf("\n%-15s %-15s ACTION\n", "STACK", "INPUT");
    printf("-----\n");

    while (input[ip] != '$')
    {
        shift();

        while (reduce())
        {
            /* Continue reducing */
        }
    }

    /* Final reductions */
    while (reduce())
    {
        /* Continue reducing */
    }

    if (top == 1 && stack[0] == '$' && stack[1] == 'E')
    {
        printf("\nInput string is ACCEPTED.\n");
    }
    else
    {
        printf("\nInput string is REJECTED.\n");
    }

    return 0;
}

```

Sample Output:

```
Command Prompt
P:\pcd_lab\exp_6>gcc shiftreduce.c -o shiftreduce.exe
P:\pcd_lab\exp_6>shiftreduce.exe
Grammar:
E -> E+E
E -> E*E
E -> id

Enter the input string: i+i*i

STACK          INPUT          ACTION
-----
$i             +i*i$             SHIFT
$E             +i*i$             REDUCE E->id
$E+            i*i$             SHIFT
$E+i           *i$              SHIFT
$E+E           *i$              REDUCE E->id
$E             *i$              REDUCE E->E+E
$E*            i$              SHIFT
$E*i           $               SHIFT
$E*E           $               REDUCE E->id
$E             $               REDUCE E->E*E

Input string is ACCEPTED.
P:\pcd_lab\exp_6>
```

```
Command Prompt
P:\pcd_lab\exp_6>shiftreduce.exe
Grammar:
E -> E+E
E -> E*E
E -> id

Enter the input string: i+*i

STACK          INPUT          ACTION
-----
$i             +*i$             SHIFT
$E             +*i$             REDUCE E->id
$E+            *i$             SHIFT
$E++           i$             SHIFT
$E++i          $             SHIFT
$E++E          $             REDUCE E->id

Input string is REJECTED.

P:\pcd_lab\exp_6>
```

Result:

Thus, the Shift Reduce Parsing Algorithm was successfully implemented using C programming, and the given input string was successfully analysed using shift and reduce operations to determine whether it was accepted or rejected.

Exp No: 7	Implementation of LR Parser
Date: 20-08-26	

Aim:

To implement an LR Parser using C programming to parse a given input string using an LR parsing table and determine whether the string is accepted or rejected.

Grammar Used

The following grammar is used:

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow id$

In the program, i is used to represent id.

Example:

i+i

represents:

id + id

Algorithm:

1. Start the program.
2. Define the grammar and LR parsing table.
3. Initialize the stack with state 0.
4. Append \$ to the input string.
5. Read the current input symbol.
6. Consult the LR parsing table using the current state and input symbol.
7. If the table contains a shift action, push the symbol and the new state onto the stack.
8. If the table contains a reduce action, replace the right-hand side of the corresponding production with its left-hand side and use the GOTO table to obtain the next state.
9. If the table contains accept, the input string is accepted.
10. If no valid action is available, the input string is rejected.
11. Continue until the input is accepted or rejected.
12. Stop the program.

LR Parsing Table:

For the grammar:

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow id$

The parsing table used is:

State	id	+	\$	GOTO E	GOTO T
0	S3			1	2
1		S4	Accept		
2		R2	R2		
3		R3	R3		
4	S3				5
5		R1	R1		

Where:

- S3 → Shift and go to state 3
- S4 → Shift and go to state 4
- R1 → Reduce by $E \rightarrow E + T$
- R2 → Reduce by $E \rightarrow T$
- R3 → Reduce by $T \rightarrow id$

Program:

```
#include <stdio.h>
#include <string.h>
```

```
char action[6][3] = {
    {'s', ' ', ' '}, // State 0
    {' ', 's', 'a'}, // State 1
    {' ', 'r', 'r'}, // State 2
    {' ', 'r', 'r'}, // State 3
    {'s', ' ', ' '}, // State 4
    {' ', 'r', 'r'}  // State 5
};
```

```
int actionNum[6][3] = {
    {3, 0, 0},
    {0, 4, 0},
    {0, 2, 2},
    {0, 3, 3},
    {3, 0, 0},
    {0, 1, 1}
};
```

```
/* GOTO table: columns E and T */
int gotoTable[6][2] = {
    {1, 2},
    {0, 0},
    {0, 0},
    {0, 0},
    {0, 5},
    {0, 0}
};
```

```
char input[100];
int stack[100];
int top = 0;
int ip = 0;
```

```
/* Get ACTION table column */
int getColumn(char symbol)
{
    if (symbol == 'i')
        return 0;

    if (symbol == '+')
        return 1;

    if (symbol == '$')
        return 2;
}
```

```

    return -1;
}

/* Convert stack contents into a string */
void getStackString(char str[])
{
    int i;

    str[0] = '\0';

    for (i = 0; i <= top; i++)
    {
        char temp[10];

        sprintf(temp, "%d ", stack[i]);
        strcat(str, temp);
    }
}

int main()
{
    int state;
    int column;
    char act;
    int number;
    char stackStr[200];

    printf("Grammar:\n");
    printf("1. E -> E+T\n");
    printf("2. E -> T\n");
    printf("3. T -> id\n\n");

    printf("Enter the input string: ");
    scanf("%s", input);

    strcat(input, "$");

    /* Initial state */
    stack[0] = 0;

    printf("\n%-20s %-15s %s\n",
        "Stack", "Input", "Action");

    printf("-----\n");

    while (1)
    {
        state = stack[top];

        column = getColumn(input[ip]);

        if (column == -1)
        {
            getStackString(stackStr);

```

```

printf("%-20s %-15s Error\n",
      stackStr, &input[ip]);

printf("\nInvalid symbol in input.\n");
printf("\nInput string is REJECTED.\n");

break;
}

act = action[state][column];
number = actionNum[state][column];

getStackString(stackStr);

/* SHIFT */
if (act == 's')
{
    char actionText[30];

    sprintf(actionText, "Shift %d", number);

    printf("%-20s %-15s %s\n",
          stackStr,
          &input[ip],
          actionText);

    top++;
    stack[top] = number;

    ip++;
}

/* REDUCE */
else if (act == 'r')
{
    char actionText[40];

    if (number == 1)
    {
        /*
         E -> E + T
         RHS contains 3 symbols
        */

        sprintf(actionText, "Reduce E -> E+T");

        printf("%-20s %-15s %s\n",
              stackStr,
              &input[ip],
              actionText);

        top = top - 3;

        state = stack[top];
    }
}

```

```

    top++;
    stack[top] = gotoTable[state][0];
}

else if (number == 2)
{
    /*
     E -> T
     RHS contains 1 symbol
    */

    sprintf(actionText, "Reduce E -> T");

    printf("%-20s %-15s %s\n",
        stackStr,
        &input[ip],
        actionText);

    top = top - 1;

    state = stack[top];

    top++;
    stack[top] = gotoTable[state][0];
}

else if (number == 3)
{
    /*
     T -> id
     RHS contains 1 symbol
    */

    sprintf(actionText, "Reduce T -> id");

    printf("%-20s %-15s %s\n",
        stackStr,
        &input[ip],
        actionText);

    top = top - 1;

    state = stack[top];

    top++;
    stack[top] = gotoTable[state][1];
}
}

/* ACCEPT */
else if (act == 'a')
{
    printf("%-20s %-15s Accept\n",
        stackStr,

```

```
        &input[ip]);

printf("\nInput string is ACCEPTED.\n");

break;
}

/* ERROR */
else
{
    printf("%-20s %-15s Error\n",
        stackStr,
        &input[ip]);

    printf("\nInput string is REJECTED.\n");

    break;
}
}

return 0;
}
```

Sample Output:

```
Command Prompt
P:\pcd_lab\exp_7>gcc lrparser.c -o lrparser.exe
P:\pcd_lab\exp_7>lrparser.exe
Grammar:
1. E -> E+T
2. E -> T
3. T -> id
Enter the input string: i+i

Stack          Input          Action
-----
0              i+i$          Shift 3
0 3            +i$          Reduce T -> id
0 2            +i$          Reduce E -> T
0 1            +i$          Shift 4
0 1 4          i$           Shift 3
0 1 4 3        $           Reduce T -> id
0 1 4 5        $           Reduce E -> E+T
0 1            $           Accept

Input string is ACCEPTED.
P:\pcd_lab\exp_7>
```

```
Command Prompt
P:\pcd_lab\exp_7>gcc lrparser.c -o lrparser.exe

P:\pcd_lab\exp_7>lrparser.exe
Grammar:
1. E -> E+T
2. E -> T
3. T -> id

Enter the input string: i+

Stack          Input          Action
-----
0              i+$          Shift 3
0 3            +$          Reduce T -> id
0 2            +$          Reduce E -> T
0 1            +$          Shift 4
0 1 4          $          Error

Input string is REJECTED.

P:\pcd_lab\exp_7>
```

Result:

Thus, the LR Parser was successfully implemented using C programming, and the given input string was successfully parsed using the LR parsing table. Valid strings were accepted and invalid strings were rejected.

Exp No: 8	Implementation of Front End of a Compiler
Date: 27-08-26	

Aim:

To implement the front end of a compiler that generates three-address code for arithmetic expressions using an intermediate code generation technique.

Algorithm:

1. Start the program.
2. Read the input arithmetic expression.
3. Separate the destination variable and the arithmetic expression.
4. Initialize the temporary variable counter.
5. Scan the expression and identify operands and operators.
6. Apply operator precedence to determine the order of operations.
7. Generate three-address code for each operation using temporary variables.
8. Replace evaluated expressions with temporary variables.
9. Generate the final assignment statement.
10. Display the generated three-address code.
11. Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char operands[50][20];
char operators[50];
int operandTop = -1, operatorTop = -1;
int temp = 1;

int precedence(char op)
{
    if (op == '*' || op == '/')
        return 2;

    if (op == '+' || op == '-')
        return 1;

    return 0;
}

void generateCode()
{
    char op1[20], op2[20], result[20];
    char op = operators[operatorTop--];

    strcpy(op2, operands[operandTop--]);
    strcpy(op1, operands[operandTop--]);

    sprintf(result, "%d", temp++);
}
```

```

printf("%s = %s %c %s\n",
    result, op1, op, op2);

strcpy(operands[++operandTop], result);
}

int main()
{
    char exp[100];
    char lhs[20];
    char rhs[100];
    char *equal;
    int i;

    printf("Enter expression: ");
    scanf("%99s", exp);

    equal = strchr(exp, '=');

    if (equal == NULL)
    {
        printf("Invalid expression: Assignment missing.\n");
        return 0;
    }

    /* Extract destination variable */
    strncpy(lhs, exp, equal - exp);
    lhs[equal - exp] = '\0';

    /* Extract right-hand-side expression */
    strcpy(rhs, equal + 1);

    if (lhs[0] == '\0' || rhs[0] == '\0')
    {
        printf("Invalid expression.\n");
        return 0;
    }

    printf("\nThree Address Code:\n");

    for (i = 0; rhs[i] != '\0'; i++)
    {
        /* Ignore spaces */
        if (isspace((unsigned char)rhs[i]))
            continue;

        /* Operand */
        if (isalnum((unsigned char)rhs[i]))
        {
            char operand[20];
            int j = 0;

            while (isalnum((unsigned char)rhs[i]))
                operand[j++] = rhs[i++];

```

```

    operand[j] = '\0';
    i--;

    strcpy(operands[++operandTop], operand);
}

/* Opening parenthesis */
else if (rhs[i] == '(')
{
    operators[++operatorTop] = rhs[i];
}

/* Closing parenthesis */
else if (rhs[i] == ')')
{
    while (operatorTop >= 0 &&
           operators[operatorTop] != '(')
    {
        generateCode();
    }

    if (operatorTop >= 0)
        operatorTop--;
    else
    {
        printf("Invalid expression.\n");
        return 0;
    }
}

/* Arithmetic operators */
else if (rhs[i] == '+' || rhs[i] == '-' ||
         rhs[i] == '*' || rhs[i] == '/')
{
    while (operatorTop >= 0 &&
           operators[operatorTop] != '(' &&
           precedence(operators[operatorTop]) >=
           precedence(rhs[i]))
    {
        generateCode();
    }

    operators[++operatorTop] = rhs[i];
}

else
{
    printf("Invalid symbol in expression.\n");
    return 0;
}
}

/* Generate remaining operations */
while (operatorTop >= 0)

```

```
{
    if (operators[operatorTop] == '(')
    {
        printf("Invalid expression.\n");
        return 0;
    }

    generateCode();
}

if (operandTop != 0)
{
    printf("Invalid expression.\n");
    return 0;
}

/* Final assignment */
printf("%s = %s\n", lhs, operands[operandTop]);

return 0;
}
```

Sample Output:

```
Command Prompt

P:\pcd_lab\exp_8>gcc frontend.c -o frontend.exe

P:\pcd_lab\exp_8>frontend.exe
Enter expression: a=b+c*d

Three Address Code:
t1 = c * d
t2 = b + t1
a = t2

P:\pcd_lab\exp_8>
```

```
Command Prompt

P:\pcd_lab\exp_8>gcc frontend.c -o frontend.exe

P:\pcd_lab\exp_8>frontend.exe
Enter expression: result=p+q*r-s

Three Address Code:
t1 = q * r
t2 = p + t1
t3 = t2 - s
result = t3

P:\pcd_lab\exp_8>
```

Result:

Thus, the front end of a compiler was successfully implemented using C programming, and three-address code was generated for arithmetic expressions according to operator precedence.

Exp No: 9	IMPLEMENTATION OF COMPILER BACKEND (THREE ADDRESS CODE TO 8086 ASSEMBLY)
Date: 02-09-26	

Aim:

To implement the backend phase of a compiler that converts Three Address Code (TAC) into 8086 assembly-like instructions.

Algorithm:

1. Start.
2. Read the number of TAC instructions, n.
3. For each instruction, read the TAC in the form res = op1 operator op2.
4. Extract the result, first operand, operator, and second operand.
5. Generate the instruction MOV op1, R0.
6. Depending on the operator, generate the corresponding instruction:
 - a. + → ADD op2, R0
 - b. - → SUB op2, R0
 - c. * → MUL op2, R0
 - d. / → DIV op2, R0
7. Generate MOV R0, res.
8. Repeat the process for all TAC instructions.
9. Stop.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

struct tac
{
    char res[10], op1[10], op2[10], op;
};

int main()
{
    struct tac t[20];
    int n;
    char input[50];

    printf("Enter number of TAC instructions: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        printf("Enter TAC %d (ex: t1=c*60): ", i + 1);
        scanf("%49s", input);

        int j = 0, k = 0;

        // Extract result
        while (input[j] != '=' && input[j] != '\0')
            t[i].res[k++] = input[j++];
```

```

t[i].res[k] = '\0';
j++; // Skip '='

// Extract first operand
k = 0;
while (input[j] != '+' && input[j] != '-' &&
      input[j] != '*' && input[j] != '/' &&
      input[j] != '\0')
    t[i].op1[k++] = input[j++];

t[i].op1[k] = '\0';

// Extract operator
t[i].op = input[j++];

// Extract second operand
k = 0;
while (input[j] != '\0')
    t[i].op2[k++] = input[j++];

t[i].op2[k] = '\0';
}

printf("\n--- Assembly Code ---\n");

for (int i = 0; i < n; i++)
{
    printf("MOV %s, R0\n", t[i].op1);

    switch (t[i].op)
    {
        case '+':
            printf("ADD %s, R0\n", t[i].op2);
            break;

        case '-':
            printf("SUB %s, R0\n", t[i].op2);
            break;

        case '*':
            printf("MUL %s, R0\n", t[i].op2);
            break;

        case '/':
            printf("DIV %s, R0\n", t[i].op2);
            break;

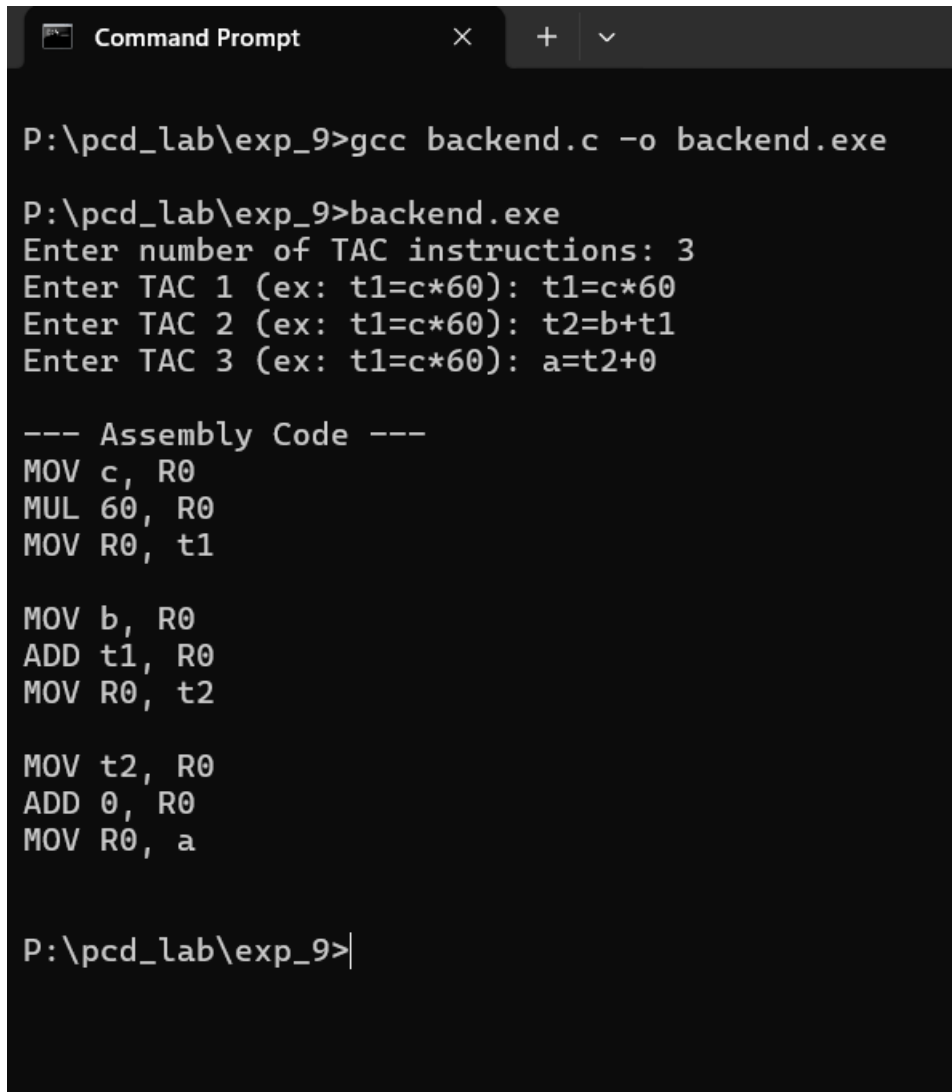
        default:
            printf("Invalid operator\n");
            break;
    }

    printf("MOV R0, %s\n\n", t[i].res);
}

```

```
    return 0;  
}
```

Sample Output:



```
Command Prompt  
P:\pcd_lab\exp_9>gcc backend.c -o backend.exe  
  
P:\pcd_lab\exp_9>backend.exe  
Enter number of TAC instructions: 3  
Enter TAC 1 (ex: t1=c*60): t1=c*60  
Enter TAC 2 (ex: t1=c*60): t2=b+t1  
Enter TAC 3 (ex: t1=c*60): a=t2+0  
  
--- Assembly Code ---  
MOV c, R0  
MUL 60, R0  
MOV R0, t1  
  
MOV b, R0  
ADD t1, R0  
MOV R0, t2  
  
MOV t2, R0  
ADD 0, R0  
MOV R0, a  
  
P:\pcd_lab\exp_9>
```

```
Command Prompt
P:\pcd_lab\exp_9>gcc backend.c -o backend.exe

P:\pcd_lab\exp_9>backend.exe
Enter number of TAC instructions: 3
Enter TAC 1 (ex: t1=c*60): t1=a+b
Enter TAC 2 (ex: t1=c*60): t2=t1*c
Enter TAC 3 (ex: t1=c*60): result=t2-d

--- Assembly Code ---
MOV a, R0
ADD b, R0
MOV R0, t1

MOV t1, R0
MUL c, R0
MOV R0, t2

MOV t2, R0
SUB d, R0
MOV R0, result

P:\pcd_lab\exp_9>
```

Result:

Thus, the given Three Address Code was successfully converted into equivalent assembly-like instructions using the backend phase of the compiler.

Exp No: 10	Implementation of Code Optimization
Date: 10-09-26	

Aim:

To implement the code optimization phase of a compiler that eliminates dead code and common subexpressions from a given set of statements.

Algorithm:

1. Start.
2. Read the number of expressions.
3. Read each expression in three-address form.
4. For Dead Code Elimination:
 - a. Identify statements whose results are never used.
 - b. Remove those statements.
5. For Common Subexpression Elimination:
 - a. Compare the expressions.
 - b. If the same expression appears again, replace it with the previously computed value.
6. Display the optimized code.
7. Stop.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

struct code
{
    char lhs[10], op1[10], op2[10], op[5];
} c[20];

int main()
{
    int n, i, j;
    int flag[20] = {0};

    printf("Enter number of expressions: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter expression %d (lhs op1 operator op2): ",
              i + 1);

        scanf("%9s %9s %4s %9s",
              c[i].lhs, c[i].op1, c[i].op, c[i].op2);
    }

    printf("\n--- Optimized Code ---\n");

    for (i = 0; i < n; i++)
    {
```

```

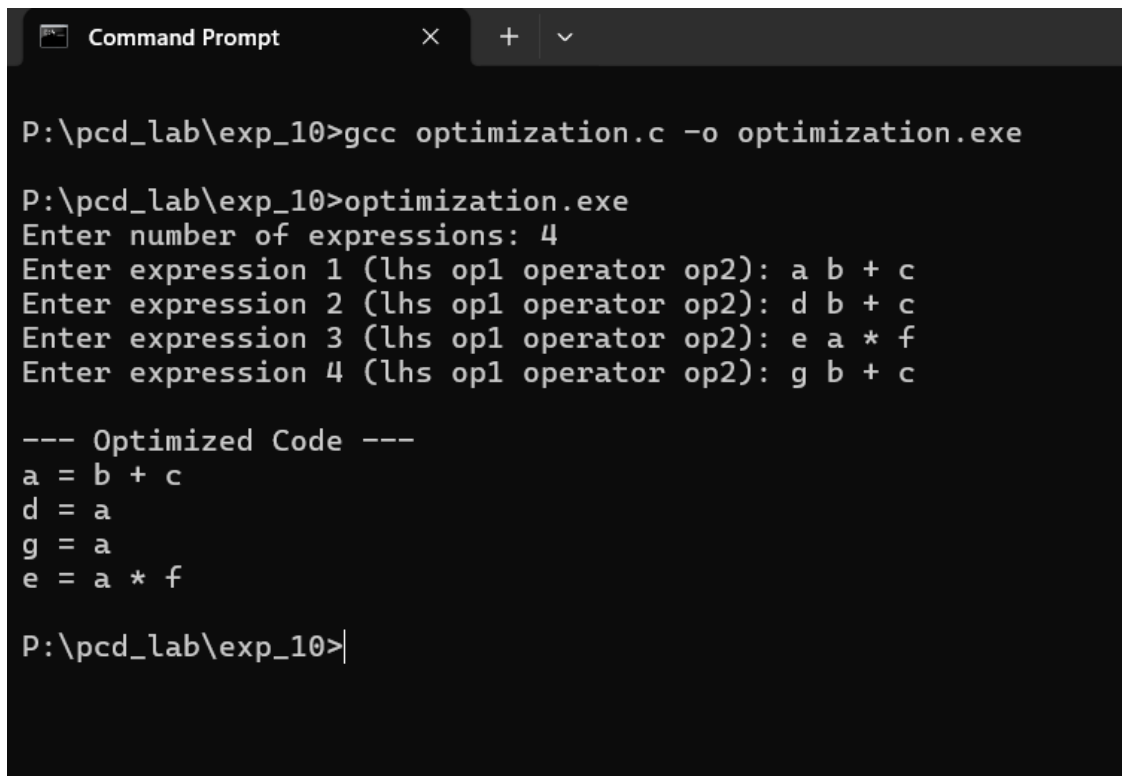
if (flag[i] == 0)
{
    printf("%s = %s %s %s\n",
        c[i].lhs, c[i].op1, c[i].op, c[i].op2);

    for (j = i + 1; j < n; j++)
    {
        if (strcmp(c[i].op1, c[j].op1) == 0 &&
            strcmp(c[i].op2, c[j].op2) == 0 &&
            strcmp(c[i].op, c[j].op) == 0)
        {
            printf("%s = %s\n", c[j].lhs, c[i].lhs);
            flag[j] = 1;
        }
    }
}
}

return 0;
}

```

Sample Output:



```

P:\pcd_lab\exp_10>gcc optimization.c -o optimization.exe

P:\pcd_lab\exp_10>optimization.exe
Enter number of expressions: 4
Enter expression 1 (lhs op1 operator op2): a b + c
Enter expression 2 (lhs op1 operator op2): d b + c
Enter expression 3 (lhs op1 operator op2): e a * f
Enter expression 4 (lhs op1 operator op2): g b + c

--- Optimized Code ---
a = b + c
d = a
g = a
e = a * f

P:\pcd_lab\exp_10>

```

```
Command Prompt
P:\pcd_lab\exp_10>gcc optimization.c -o optimization.exe

P:\pcd_lab\exp_10>optimization.exe
Enter number of expressions: 4
Enter expression 1 (lhs op1 operator op2): x p * q
Enter expression 2 (lhs op1 operator op2): y p * q
Enter expression 3 (lhs op1 operator op2): z x + r
Enter expression 4 (lhs op1 operator op2): w p * q

--- Optimized Code ---
x = p * q
y = x
w = x
z = x + r

P:\pcd_lab\exp_10>
```

Result:

Thus, common subexpressions were identified and redundant computations were replaced with previously computed values, reducing repeated calculations.